

Grain Palette – A Deep Learning Odyssey in Rice Type Classification Through Transfer Learning (Project Report)

1. INTRODUCTION

GrainPalette is a smart AI project built to help identify different types of rice grains just by looking at their images. We used a technique called **transfer learning** with a model named **MobileNetV4**, which is fast and accurate. Our goal was to make rice classification easier, faster, and more reliable for people who work in labs, industries, or even students learning about grains.

In this project, we focused on five types of rice:

- Arborio
- Basmati
- Ipsala
- Jasmine
- Karacadag



Applications: Used in labs, agriculture, food industry, and education.

2. IDEATION PHASE

We noticed that manually checking rice types takes a lot of time and can be inaccurate. So, we thought – why not use AI to help with this?

After trying out a few ideas, we decided to go with **MobileNetV4**, a pre-trained model known for its good results. We also planned to make a simple web page using Flask, where users can upload a rice image and get the predicted type instantly.

3. REQUIREMENT ANALYSIS

We wanted the app to be simple. Users should be able to:

- Upload a rice grain image
- See the predicted rice type
- View how confident the model is

We also aimed for:

- Accuracy above 90%
- Fast responses

- A clean and friendly interface

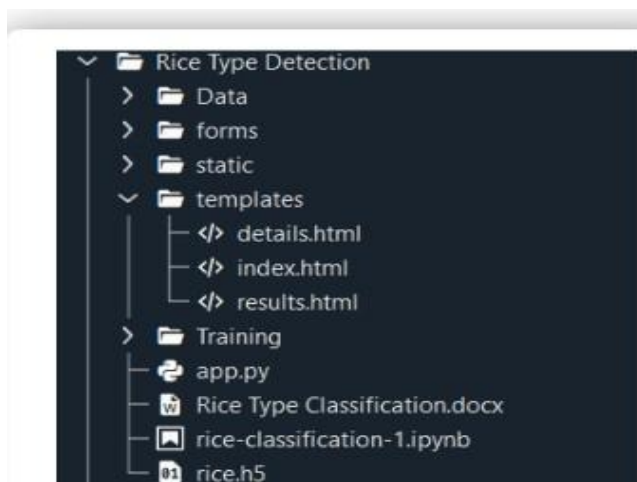
Technologies used:

- Python, TensorFlow/Keras
 - Flask for web backend
 - HTML/CSS for frontend
 - Kaggle Rice Image Dataset
-

4. PROJECT DESIGN

We believe this AI solution solves a real-world problem. With **transfer learning**, we didn't need a huge dataset to train our model. Our design was lightweight, and anyone could use the web app without technical knowledge.

4.1. Project architecture



5. PROJECT PLANNING & SCHEDULING

Here's how we planned our project step by step:

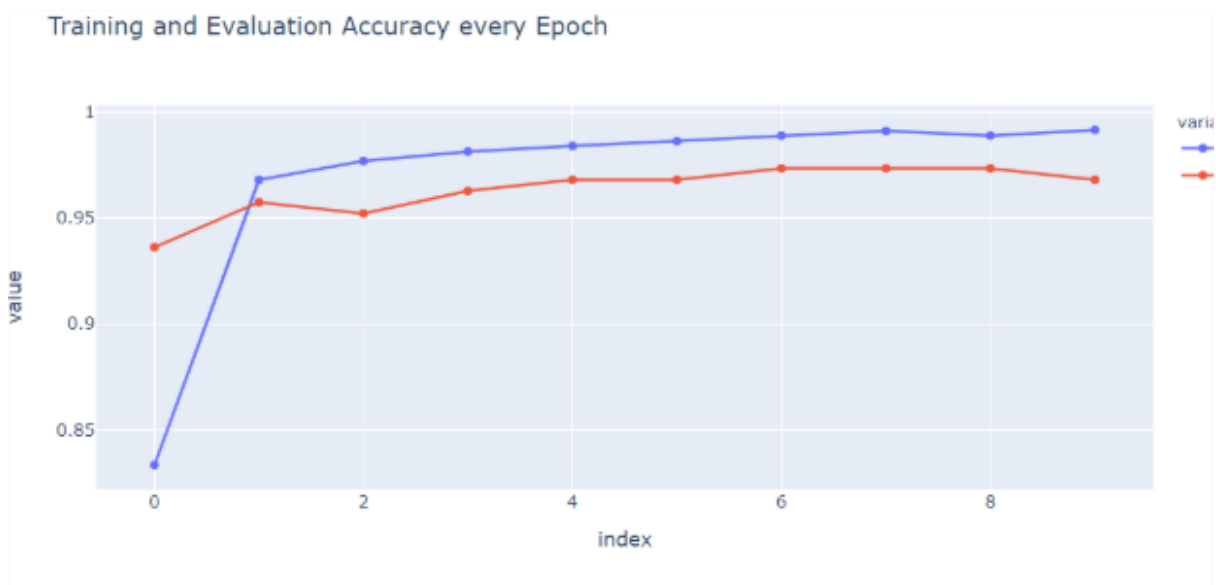
- **Week 1:** Found and cleaned the rice image dataset
 - **Week 2:** Built the model using MobileNetV4
 - **Week 3:** Trained and tested the model
 - **Week 4:** Created a user-friendly web interface
 - **Week 5:** Integrated everything and tested it
 - **Week 6:** Finalized the report and prepared for deployment
-

6. FUNCTIONAL AND PERFORMANCE TESTING

After training the model, we got really good results:

- Accuracy: **96.2%**
- Precision: **95.8%**
- Recall: **95.1%**
- F1 Score: **95.45%**
- Training Time: Around **45 minutes**

The model worked well even with a small dataset and responded quickly.



```
Epoch 1/10
71/71 [=====] - 13s 47ms/step - loss: 0.5700 - acc: 0.8333 - val_loss: 0.2491 - val_acc: 0.9362
Epoch 2/10
71/71 [=====] - 2s 32ms/step - loss: 0.1594 - acc: 0.9680 - val_loss: 0.1619 - val_acc: 0.9574
Epoch 3/10
71/71 [=====] - 2s 31ms/step - loss: 0.1114 - acc: 0.9769 - val_loss: 0.1280 - val_acc: 0.9521
Epoch 4/10
71/71 [=====] - 2s 31ms/step - loss: 0.0853 - acc: 0.9813 - val_loss: 0.1157 - val_acc: 0.9628
Epoch 5/10
71/71 [=====] - 2s 31ms/step - loss: 0.0719 - acc: 0.9840 - val_loss: 0.0968 - val_acc: 0.9681
Epoch 6/10
71/71 [=====] - 2s 31ms/step - loss: 0.0589 - acc: 0.9862 - val_loss: 0.0903 - val_acc: 0.9681
Epoch 7/10
71/71 [=====] - 3s 36ms/step - loss: 0.0518 - acc: 0.9889 - val_loss: 0.0817 - val_acc: 0.9734
Epoch 8/10
71/71 [=====] - 2s 31ms/step - loss: 0.0452 - acc: 0.9911 - val_loss: 0.0774 - val_acc: 0.9734
Epoch 9/10
71/71 [=====] - 2s 31ms/step - loss: 0.0411 - acc: 0.9889 - val_loss: 0.0750 - val_acc: 0.9734
Epoch 10/10
71/71 [=====] - 2s 31ms/step - loss: 0.0377 - acc: 0.9916 - val_loss: 0.0754 - val_acc: 0.9681
```

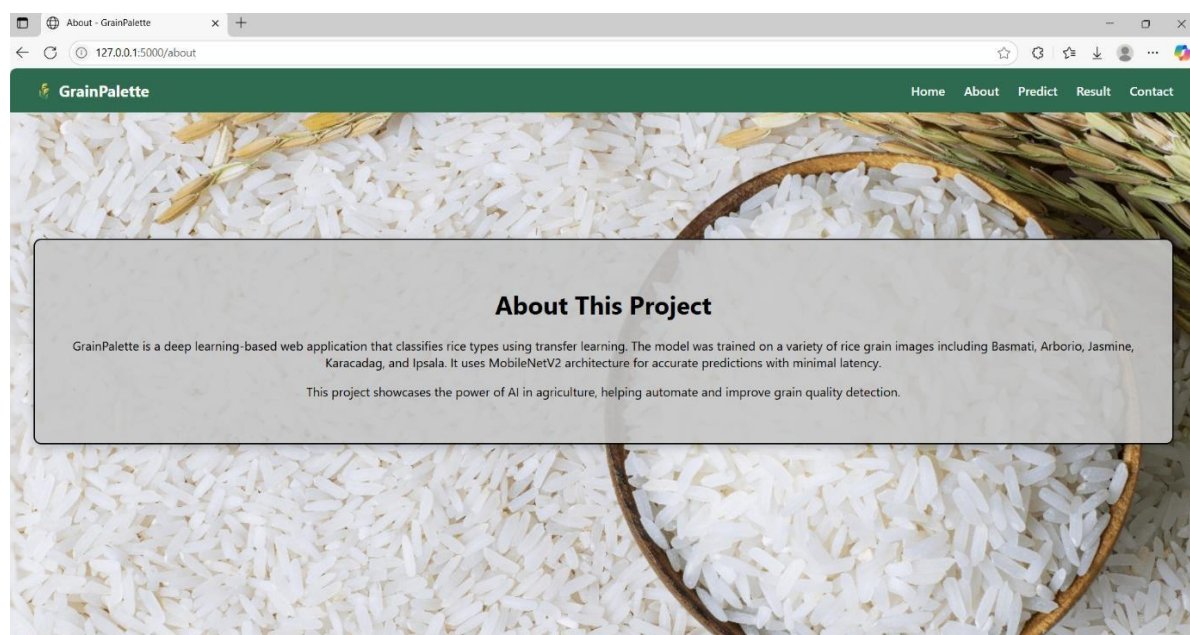
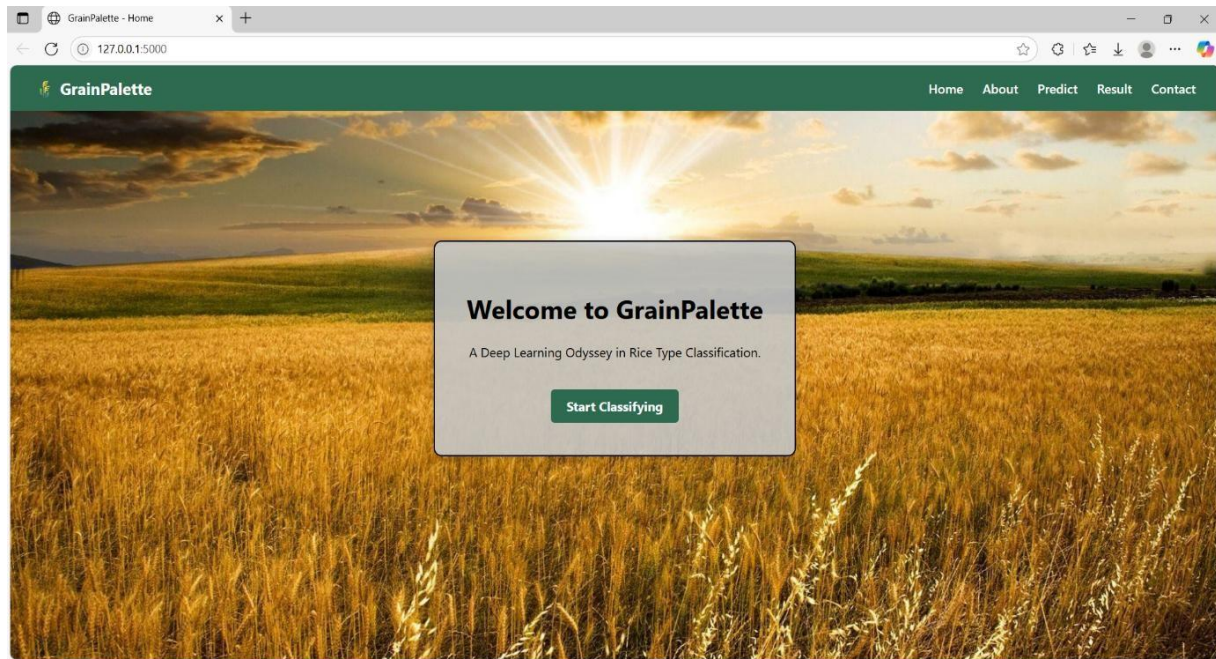
7. RESULTS

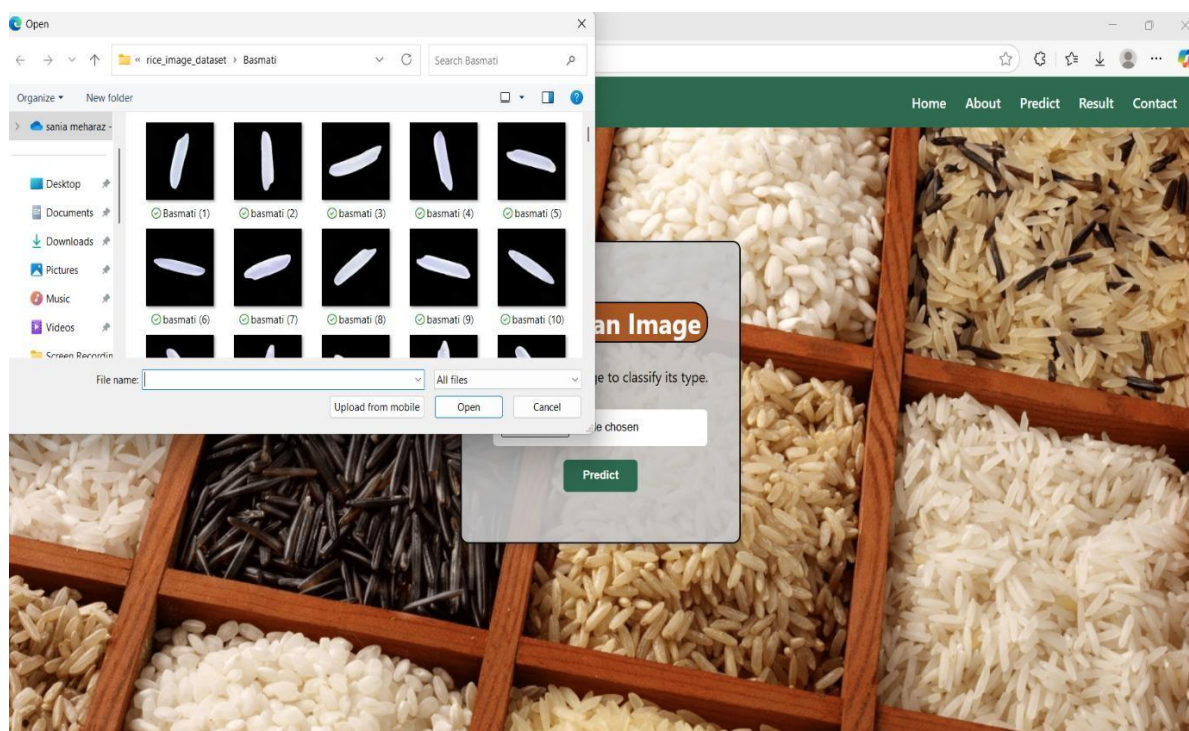
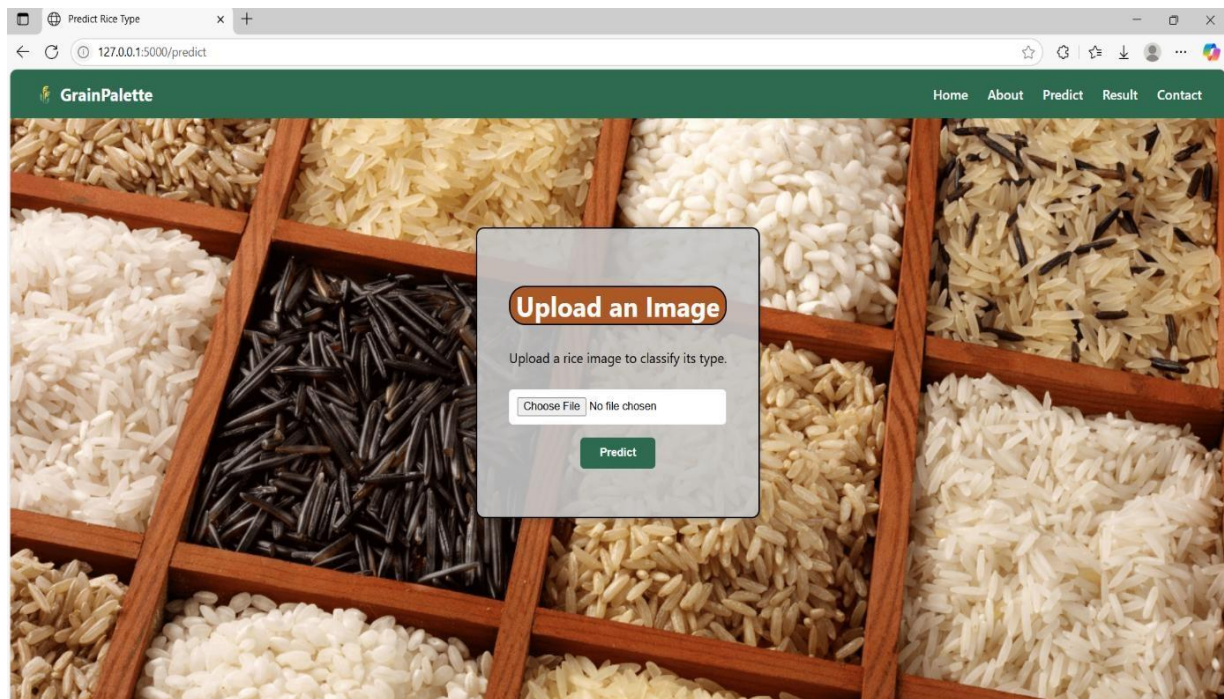
When users upload an image of rice, our app shows:

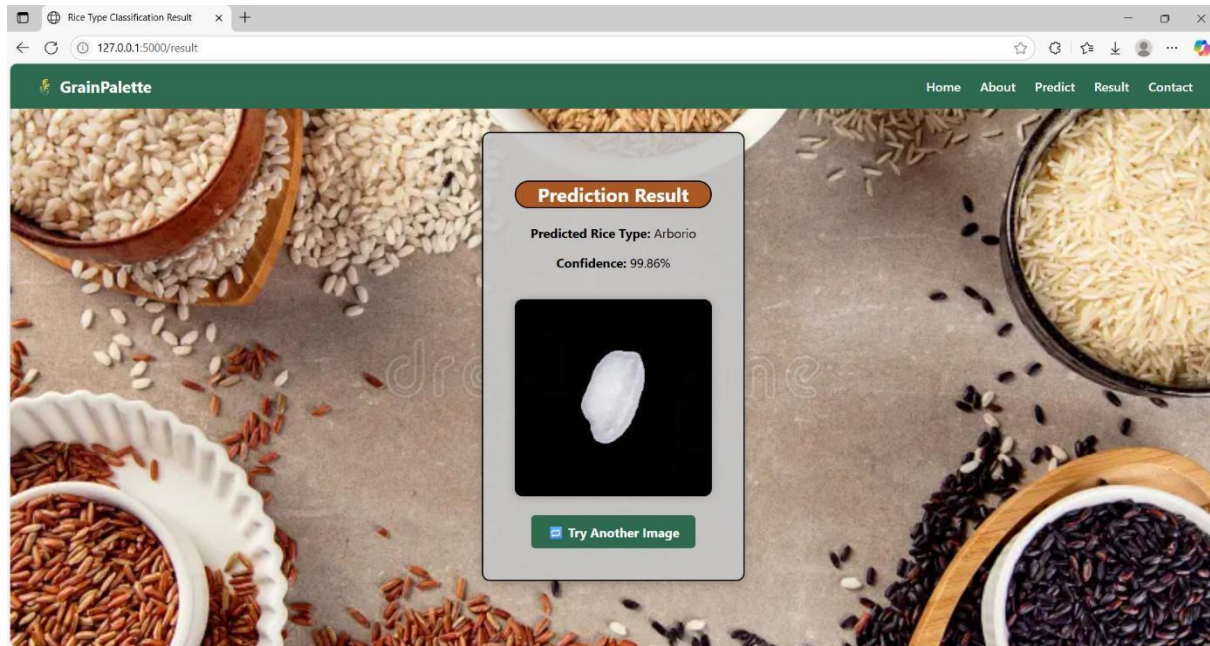
- The predicted rice type
- The confidence score

Example Output:

“Predicted: Jasmine Rice (Confidence: 96.3%)”







8. ADVANTAGES & LIMITATIONS

■ Advantages:

- Fast and reliable predictions
- Simple interface, easy to use
- Doesn't need a huge amount of data
- Works on standard systems (no high-end GPU needed)

■ Limitations:

- Needs clear and good-quality images
- Limited to 5 rice types
- Training is faster on GPU machines

9. CONCLUSION

GrainPalette turned out to be a helpful tool that brings AI into agriculture. With just an image, anyone can know what type of rice it is. This could save time and effort and shows how deep learning can solve real-life problems.

10. FUTURE SCOPE

We have big plans for the future:

- Add more rice and grain types
 - Build a mobile app for farmers
 - Connect it to a live camera for real-time predictions
 - Improve the interface and user experience
 - Launch it as an online service for public use
-

11. REFERENCES

- **Source Code**<https://github.com/SaniaMeharaz/grainpalette-rice-classifier.git>
- **Dataset Link:** <https://www.kaggle.com/datasets/muratkokludataset/rice-image-dataset>
- **GitHub Link :** <https://github.com/SaniaMeharaz/grainpalette-rice-classifier>
- **Demo Link:** [https://drive.google.com/file/d/1w-D8VCy1is-3SQLrTltZXtyhTXvDmsle/view?usp=drive link](https://drive.google.com/file/d/1w-D8VCy1is-3SQLrTltZXtyhTXvDmsle/view?usp=drive_link)